中国科学院大学
《计算机组成原理(研讨课)》实验报告

姓名 王浚沣 学号 2024K8009909018 专业 计算机科学与技术

实验项目编号 4 实验名称 DNN 加速与定制指令设计

注 1:	撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下 (注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中 "-" 为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
注 2:	使用 `git add` 及 `git commit` 命令将实验报告 `PDF` 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
注 3:	实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

• **设计**步骤为在代码编写前的准备阶段, 主要包括卷积/池化算法分析、定点数格式选择、乘法指令扩展和硬件加速器调用方式设计。

• **开发**步骤为在 S-IDE 集成开发环境中进行 C/Verilog 代码编写, 实现软件 DNN 推理流程以及 CPU 中的乘法指令支持。

• **调试**步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 并通过输出结果、周期数和指令数判断实现是否正确。

一、 设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 LaTeX 中)

1 DNN 软件流程设计

本次实验的目标是完成一个简单 DNN 计算任务, 主要包括卷积和池化两个阶段。输入数据、权重数据和输出数据分别存放在指定内存地址中, 程序需要从输入区域读取图像特征图, 从权重区域读取卷积核和 bias, 完成卷积后再进行最大池化, 最后将结果与标准答案进行比较。

卷积部分采用四维张量的思路组织数据。输入张量大小为 $1 \times 1 \times 28 \times 28$, 权重大小为 $20 \times 1 \times 5 \times 5$, 输出卷积结果大小为 $1 \times 20 \times 24 \times 24$。由于后续还要进行池化, 最终输出大小为 $1 \times 20 \times 12 \times 12$。代码中使用 'size_vec4' 结构体保存各维度大小, 用 'mem_addr' 结构体保存输入、权重和输出地址。这样的组织方式比直接写死一串地址偏移更清晰, 也方便在计算 offset 时检查每一维的含义。

卷积计算的核心是五层循环:batch、输出通道、输出高度、输出宽度、输入通道以及卷积核内部的高度和宽度。在每一个输出位置, 程序先读取当前输出通道对应的 bias, 并将其左移 'FRAC_BIT' 位作为累加初值。随后遍历输入通道和卷积核窗口, 将输入数据与对应权重相乘并累加。由于实验中使用定点数表示, 最后需要将累加结果右移 'FRAC_BIT' 位, 得到最终写入输出数组的 short 类型结果。

2 池化与结果比较设计

池化部分采用 2×2 最大池化, stride 为 2。它直接在卷积输出所在的 'WR_ADDR' 区域上进行原地操作, 也就是读写使用同一片输出内存。由于卷积输出大小为 24×24, 池化后每个输出通道变为 12×12。池化过程同样按照 batch、通道、输出高度、输出宽度遍历, 并在每个窗口内寻找最大值。

在实现时,我没有直接假设所有窗口都完全落在输入范围内,而是仍然保留了 pad 和边界判断逻辑。这样虽然本实验参数中 pad 为 0,但代码结构更接近通用池化实现。每个窗口中合法位置的数据会与当前最大值比较,最终把最大值写回新的输出位置。

结果比较阶段通过 '_binary_data_result_bin_start' 和 '_binary_data_result_bin_size' 找到标准答案,并逐字节与输出区域比较。如果启用了硬件加速,由于硬件输出可能存在对齐填充,还需要跳过每行尾部的无效对齐区域。比较通过后输出 Passed,否则打印出错地址和数据。这个设计使调试时可以直接定位到第一个不一致的位置。

3 CPU 定制乘法指令设计

为了提高 DNN 计算中的乘法效率,我在 CPU 中加入了乘法指令支持。原本软件可以调用 'mul_ll' 完成乘法,但在卷积中乘法出现次数非常多,如果完全依赖软件函数,会产生较大的指令开销。因此我在 Verilog 中新增了 'Mul' 指令类型,并通过 opcode、shamt 和 func 字段识别对应的乘法指令。

乘法模块的实现方式比较直接:使用 'RF_rdata1 * RF_rdata2' 得到 64 bit 乘积,然后取低 32 bit 作为写回结果。由于本实验中的卷积数据为 short 类型,乘法结果再经过定点数缩放后使用,因此低 32 bit 已经可以满足当前计算需求。寄存器写回数据选择逻辑中也加入了 'mode == Mul' 的分支,使乘法结果能够像普通 ALU 结果一样写回寄存器堆。

这种扩展方式没有引入新的流水级或复杂握手机制,而是把乘法作为一个组合运算加入原有执行流程。实现简单,调试成本较低,也能直观体现定制指令对热点计算的加速作用。

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 卷积函数开发

卷积函数首先将输入、权重和输出地址转换为 short 指针,然后根据输入尺寸、权重尺寸、padding 和 stride 计算卷积输出尺寸。由于输出尺寸后续还会被池化函数使用,因此我将其保存到全局的 'conv_size' 结构体中。

偏移计算是卷积函数中最容易写错的部分。输入数据的 offset 由 batch、输入通道、输入高度和输入宽度共同决定; 权重 offset 则需要先定位到输出通道对应的 filter, 再根据输入通道和卷积核内部位置找到具体权重。代码中我先计算 'input_channel_size'、'input_batch_size'、'output_channel_size'、'output_batch_size'、'kernel_area'、'kernel_block' 和 'filter_block' 等中间量,避免在内层循环中反复写很长的表达式。

卷积的 bias 放在每个 filter block 的开头,因此 'weight[filter_base]' 即为当前输出通道的偏置项。真正的卷积核权重从 'channel_weight_base + 1' 开始。累加时我使用 int 类型保存 sum,避免 short 乘法累加过程中溢出过早截断。最后写回时再右移定点小数位,并转换为 short。

```
unsigned input_channel_size = mul(input_fm_h, input_fm_w);
unsigned input_batch_size = mul(rd_size.d1, input_channel_size);
unsigned output_channel_size = mul(conv_out_h, conv_out_w);
unsigned output_batch_size = mul(conv_size.d1, output_channel_size);
unsigned kernel_area = mul(weight_size.d2, weight_size.d3);
unsigned kernel_block = kernel_area + 1;
unsigned filter_block = mul(rd_size.d1, kernel_block);

for (unsigned n = 0; n < rd_size.d0; n++){
    for (unsigned oc = 0; oc < conv_size.d1; oc++){
        unsigned filter_base = mul(oc, filter_block);
        int bias = weight[filter_base];

        for (unsigned oh = 0; oh < conv_out_h; oh++){
            for (unsigned ow = 0; ow < conv_out_w; ow++){
                int sum = bias << FRAC_BIT;

                for (unsigned ic = 0; ic < rd_size.d1; ic++){
                    unsigned channel_weight_base = filter_base + mul(ic, kernel_block);

                    for (unsigned kh = 0; kh < weight_size.d2; kh++){
                        for (unsigned kw = 0; kw < weight_size.d3; kw++){
                            int ih = (int)mul(oh, stride) + kh - pad;
                            int iw = (int)mul(ow, stride) + kw - pad;

                            if (ih >= 0 && ih < input_fm_h && iw >= 0 && iw < input_fm_w)
                            {
                                input_offset =
                                    mul(n, input_batch_size) +
                                    mul(ic, input_channel_size) +
                                    mul(ih, input_fm_w) +
                                    iw;

                                unsigned weight_offset =
                                    channel_weight_base +
                                    1 +
                                    mul(kh, weight_size.d3) +
                                    kw;

                                sum += mul(in[input_offset], weight[weight_offset]);
```

图 1: convolution 函数核心代码截图

2 池化函数开发

池化函数的输入就是卷积阶段写入的输出区域,因此它使用 ‘addr.wr_addr‘ 作为读写指针。根据 ‘conv_size‘ 可以得到池化输入的宽和高,再结合池化核大小、stride 和 padding 计算池化输出尺寸。

最大池化实现中,我用 ‘short max_val = (short)0x8000‘ 作为初始最小值。每个输出位置对应一个 2×2 窗口,程序遍历窗口中每个合法坐标,并从输出数组中读取对应卷积结果进行比较。计算完成后,再根据新的输出尺寸将最大值写回到输出数组前部。

```
unsigned input_channel_size = mul(input_fm_h, input_fm_w);
unsigned input_batch_size = mul(conv_size.d1, input_channel_size);
unsigned output_channel_size = mul(pool_out_h, pool_out_w);
unsigned output_batch_size = mul(conv_size.d1, output_channel_size);

for (unsigned n = 0; n < conv_size.d0; n++){
    for (unsigned oc = 0; oc < conv_size.d1; oc++){
        for (unsigned oh = 0; oh < pool_out_h; oh++){
            for (unsigned ow = 0; ow < pool_out_w; ow++){
                short max_val = (short)0x8000;

                for (unsigned kh = 0; kh < KERN_ATTR_POOL_KERN_SIZE; kh++){
                    for (unsigned kw = 0; kw < KERN_ATTR_POOL_KERN_SIZE; kw++){
                        int ih = (int)mul(oh, stride) + kh - pad;
                        int iw = (int)mul(ow, stride) + kw - pad;

                        if (ih >= 0 && ih < input_fm_h && iw >= 0 && iw < input_fm_w){
                            input_offset =
                                mul(n, input_batch_size) +
                                mul(oc, input_channel_size) +
                                mul(ih, input_fm_w) +
                                iw;

                            if (out[input_offset] > max_val){
                                max_val = out[input_offset];
                            }
                        }
                    }
                }

                output_offset =
                    mul(n, output_batch_size) +
                    mul(oc, output_channel_size) +
                    mul(oh, pool_out_w) +
                    ow;

                out[output_offset] = max_val;
            }
        }
    }
}
```

图 2: pooling 函数代码截图

3 硬件加速器启动与性能统计

性能统计通过 ‘bench_prepare‘ 和 ‘bench_done‘ 完成，最后输出 total cycle 和 total inst。这样既可以验证功能是否正确，也能比较纯软件实现、定制乘法指令实现和硬件加速实现之间的性能差异。

```
int main()
{
    Result res;
    res.msec = 0;
    res.inst_cnt = 0;

    bench_prepare(&res);

#ifdef USE_HW_ACCEL
    printf("Launching task...\n");
    launch_hw_accel();
#else
    printf("starting convolution\n");
    convolution();
    printf("starting pooling\n");
    pooling();
#endif

    bench_done(&res);
    printf("total cycle %d\n", res.msec);
    printf("total inst %d\n", res.inst_cnt);

    int result = comparing();
    printf("benchmark finished\n");

    if (result == 0) {
        hit_good_trap();
    } else {
        nemu_assert(0);
    }

    return 0;
}
```

图 3: main 函数代码截图

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

表 1: 实验问题总结

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	软件实现	卷积偏移计算	输出结果与标准答案不一致	weight 中 bias 和卷积核权重的布局没有完全理清	将 filter block 拆成 bias 和各输入通道权重, 并单独计算 'channel_weight_base'
2	软件实现	定点数处理	卷积输出整体偏大或偏小	累加初值和乘法结果没有统一按 FRAC_BIT 缩放	bias 左移后参与累加, 最终结果统一右移 FRAC_BIT
3	RTL 设计	乘法指令	乘法结果没有写回寄存器	只加入了乘法计算, 没有同步修改写回使能和写回数据选择	在指令分类、RegDst、RF_wen 和 RF_wdata 中加入 Mul 类型

- **DNN 调试问题补充**。

本次实验中最花时间的是确认多维数组的地址映射。卷积和池化在数学形式上并不复杂, 但落到一维内存地址后, 很容易在某一维的乘法因子上写错。我的处理方式是先把每一层大小都定义为中间变量, 再逐项组合 offset, 而不是在循环里直接写一个很长的式子。这样调试时可以分别检查 batch、channel、height 和 width 对最终地址的贡献。

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

通过本次 DNN 实验, 我第一次比较完整地把体系结构中的定制指令、软件热点函数和硬件加速器调用联系起来。卷积中的乘法密度很高, 因此哪怕只是加入一个简单的乘法指令, 也能明显减少软件模拟乘法带来的开销。而硬件加速器则进一步把整个计算任务交给专门模块完成, CPU 只负责启动和等待结果, 这种方式更接近真实系统中的协处理器调用。

本次实验的亮点在于软件卷积和池化流程比较完整, CPU 中也加入了乘法指令扩展; 不足之处在于乘法指令实现仍然比较简单, 只取低 32 位, 没有进一步处理更复杂的有符号乘法高位结果。同时, DNN 部分目前主要针对固定输入尺寸和固定网络结构, 通用性还不够强。

表 2: 收获总结

类别	名称	体会	后续计划
软件实现	卷积和池化	理解了多维张量在一维内存中的布局, 也体会到 offset 计算对正确性的影响很大。	后续将常用 offset 计算封装成函数, 减少重复表达式。
RTL 编写	定制乘法指令	掌握了从译码、执行到写回完整加入新指令的方法。	继续扩展乘法相关指令, 支持更完整的结果和更复杂的数据类型。
系统接口	MMIO 启动加速器	理解了 CPU 通过内存映射寄存器控制硬件模块的基本方式。	后续进一步学习中断或 DMA 方式, 减少 CPU 轮询等待。

四、 实验时长

在课后,你花费了大约____12____ 小时完成此次实验,其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	2	分析卷积、池化、定点数和乘法指令扩展方式
开发	6	完成 conv.c 中核心计算和 CPU 乘法指令支持
调试	4	对比标准答案并检查性能统计输出

表 3: 实验时长